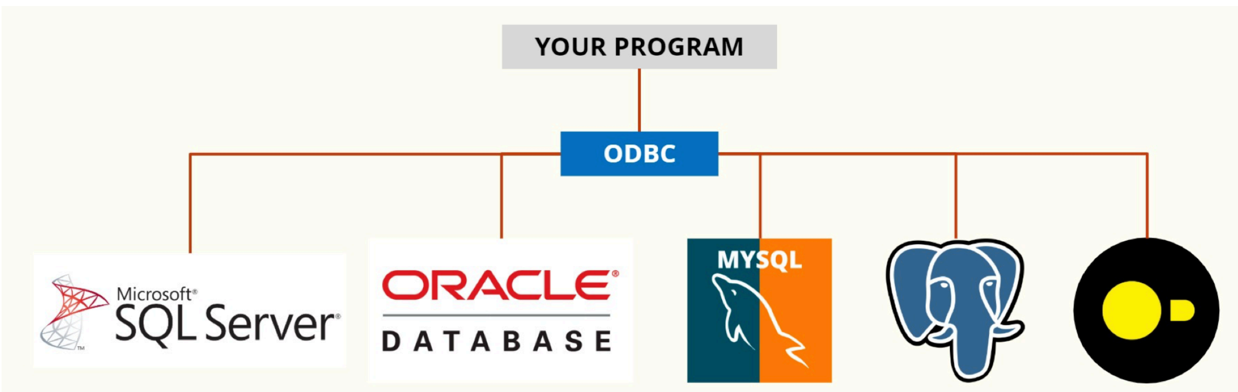
06 Programming language support

Open Database Connectivity

- In-between translation layer



Connecting DuckDB and Python

Connection

- Use ***pyodbc***
  - Package that implements ODBC and can connect to any databases

Simple database API

- **connect()**: Establish connection to specified database
- **cursor()**: Object to manage context of an SQL operation
- **execute()**: Execute SQL queries
- **fetchone()**, **fetchmany()**, **fetchall()**: Get results from queries

How to connect

```python
import duckdb

conn_string = "<local_db_file>" # or ":memory:" | "url to file"

# Connection option one
conn = duckdb.connect(conn_string, read_only=False)

# Execute Queries using Cursors
conn.close()


# Connection option two
with duckdb.connect(conn_string, read_only=False) as conn:
   # Execute Queries using Cursors
# Closes connection
```

Querying the Database

```python
# Get all athletes
athletes: list = conn.execute("SELECT * FROM Athletes").fetchall()

# Get five athletes
athletes: list = conn.execute("SELECT * FROM Athletes").fetchmany(5)

# Get one athlete
athletes: list = conn.execute("SELECT * FROM Athletes").fetchone()

# Get different number of rows from the same query
conn.execute("SELECT * FROM Athletes") # Execute query once

first_five = conn.fetchmany(5) # Fetch first 5 rows

sixth = conn.fetchone() # Fetch sixth row

the_rest = conn.fetchall() # Fetch the remaining rows
```

Executes the query and gets all athletes

Get set number of rows or a single row

Cursors

- Useful for parallel access to the database

How to get the cursor:

- cur = conn.cursor():
- cur = conn.cursor().execute(...)
- cur.fetchone(), cur.fetchall()

```python
# Get query cursor
athletes_cursor = conn.cursor().execute(”SELECT * FROM Athletes”)

# Get five athletes
athletes: list = athletes_cursor.fetchmany(5)

sports_cursor = conn.cursor().execute(”SELECT * FROM Sports”)

# Get sixth athlete
athlete_six = athletes_cursor.fetchone()

sports: list = sports_cursor.fetchall()

# Get the next 10 athletes
athlete_next_ten = athletes_cursor.fetchmany(10)

athletes_cursor.close()
sports_cursor.close()
```

Advantage: Multiple cursors active at the same time, which can fetch or execute queries

Remember to close the cursors